

Web Development Fundamentals

From a browser request to a reliable user experience

The request and response cycle

A browser requests a resource from a server and receives a response. The response may contain an HTML page, a stylesheet, a script, or structured data. URLs identify resources, while HTTP methods describe the kind of operation being requested. Reading a page and deleting a record are different actions and should use appropriate methods.

A web application connects interface design with server behavior. HTML gives content structure, CSS controls presentation, and JavaScript adds interaction. The server validates requests, enforces access rules, and reads or writes persistent data. A pleasant interface depends on all of these pieces working together.

Accessible interfaces

Semantic elements make a page easier to navigate and understand. Forms need visible labels, helpful validation messages, and keyboard-accessible controls. Buttons should describe actions, while links should lead to resources. A visible focus indicator helps people understand where keyboard interaction will happen.

Responsive layouts adapt to available space. A dashboard can use a multi-column grid on a desktop and stacked cards on a phone. Tables may need their own scroll container rather than causing the entire page to overflow. Test long filenames, empty states, loading states, and narrow viewports as part of interface design.

Authentication and data access

Authentication identifies an account, while authorization decides what that account can do. A signed-in user should not gain access to another user's documents by changing an identifier in a URL. Ownership checks belong on the server for every read, export, and deletion operation. Hiding a button is helpful presentation, but it does not enforce access.

Passwords should be stored as hashes rather than readable strings. CSRF protection helps prevent a different site from submitting actions through a user's session. Environment variables keep configuration and secrets separate from source code. Uploaded files remain untrusted even when their extension looks correct.

Building a maintainable project

Separate application features into modules with clear responsibilities. Routes coordinate a request; services implement operations; models describe persistent data. Templates render an interface without performing complex business logic. This structure makes a project easier to test and lets a developer change one layer without rewriting everything.

Document the installation steps and verify them in a fresh environment. Database migrations preserve schema history, while demo data helps reviewers explore the application quickly. Automated tests should cover successful workflows and important failure boundaries. A portfolio project feels complete when its design, behavior, documentation, and tests tell the same story.